UNIVERSIDAD DE COSTA RICA

FACULTAD DE INGENIERÍA

ESCUELA DE COMPUTACIÓN E INFORMÁTICA

Primera Entrega Programada

Curso: Programación Paralela y Concurrente

Sigla: CI-0117

Profesor: Ing. Sleyter Angulo, M.Sc.

Estudiantes:

- Jorge Antonio Durán Chinchilla, C02679.
- Kenneth Fabricio Delgado Cárdenas, C22540.
- Javier Chacón Rosales, C32052.

Semestre II
2026

1. Introducción

Este proyecto implementa un servidor de red concurrente que recibe solicitudes de simulación electromagnética y las ejecuta utilizando el método FDTD (*Finite-Difference Time-Domain*) en 2D. En la realización del motor de cálculo se paralelizó la malla bidimensional entre varios hilos y se aseguró que las actualizaciones de los campos se hicieran en el orden correcto. El problema principal radica en que cada hilo lee datos de las filas vecinas, lo que exige mecanismos rigurosos de sincronización.

El proyecto abarca desde el núcleo numérico (*solver*, barrera reutilizable, pruebas unitarias y ejecutable `fdtd_bench`) hasta la arquitectura de red *thread-per-client*, el análisis del protocolo de texto y la integración con un gestor de trabajos asíncrono. El *solver* no conoce sockets, colas ni archivos; solo calcula y notifica mediante *callbacks*. Esta separación modular permite verificar la exactitud del cálculo, medir el escalamiento del rendimiento y ofrecer una interfaz cliente-servidor tolerante a fallos.

2. Desarrollo de la solución

2.1. Representación y actualización de los campos

Los tres campos residen en arreglos separados de tipo `double`, porque en la malla de Yee tienen dimensiones distintas: E_z se ubica en los nodos, mientras que H_x y H_y se ubican en los lados. Por ello, H_x tiene $n_y - 1$ columnas y H_y posee $n_x - 1$ filas. Se utilizó `calloc` para garantizar que las condiciones iniciales fueran cero.

Las ecuaciones utilizadas para la actualización de las celdas son:

$$H_x[m][n] = H_x[m][n] - \frac{S_c}{\eta_0} \left(E_z[m][n+1] - E_z[m][n] \right),$$

$$H_y[m][n] = H_y[m][n] + \frac{S_c}{\eta_0} \left(E_z[m+1][n] - E_z[m][n] \right),$$

$$E_z[m][n] = E_z[m][n] + \frac{S_c \eta_0}{\varepsilon_r} \left((H_y[m][n] - H_y[m-1][n]) - (H_x[m][n] - H_x[m][n-1]) \right).$$

Nota: ecuaciones de actualización de celdas. Fuente: PDF Project FDTD.

La actualización eléctrica solo recorre las celdas interiores. De este modo, los bordes conservan $E_z = 0$, cumpliendo con la condición de frontera de conductor eléctrico perfecto (PEC). Posteriormente, el hilo asignado a la fila correspondiente inyecta la fuente de Ricker en (s_x, s_y).

El orden temporal es crítico: primero se actualiza H y luego E. Si se realizaran simultáneamente, E leería valores de H parcialmente actualizados, lo que introduciría condiciones de carrera. Por esta razón, se emplean dos barreras de sincronización por cada paso de tiempo.

2.2. Distribución de filas

Para T hilos, la malla se divide en T bloques contiguos de filas. La asignación es proporcional: cada hilo recibe aproximadamente n_x/T filas. Si n_x no es divisible entre T, la carga se distribuye equitativamente, dando una fila adicional a los primeros bloques. Por ejemplo, con 10 filas y 3 hilos, la distribución resulta en bloques de 3, 3 y 4 filas.

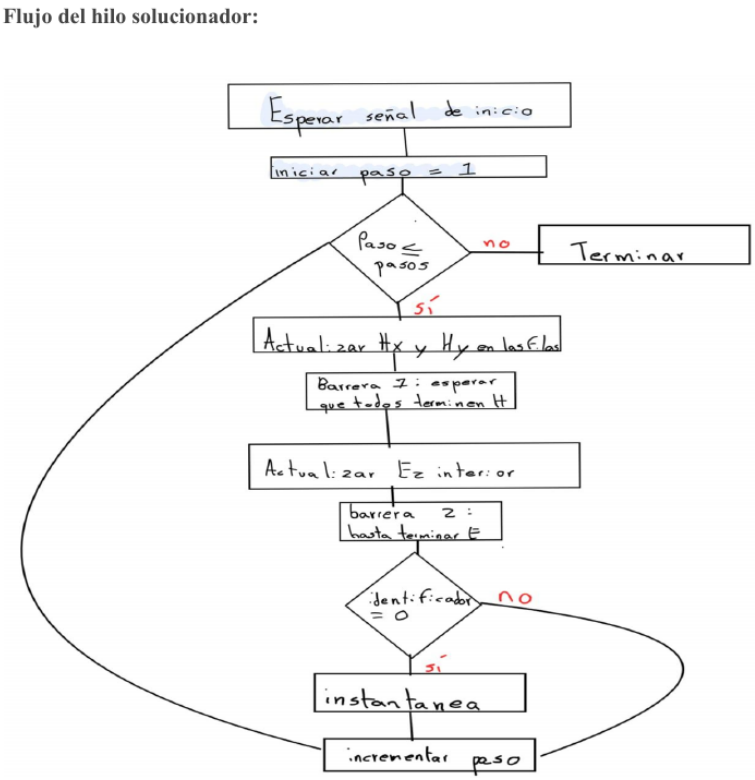


Figura 1: Flujo del hilo solucionador.

2.3. Barrera reutilizable

La barrera se implementa mediante un `pthread_mutex_t`, un contador de llegadas y dos semáforos POSIX (`sem_t`) inicializados en cero. La variable `phase` alterna entre los semáforos de cada generación.

Cada hilo que llega realiza las siguientes operaciones:

1. Adquiere el mutex, incrementa el contador y libera el mutex.
2. Si no es el último hilo en llegar, se bloquea en el semáforo correspondiente a la fase actual.
3. Si es el último hilo, reinicia el contador, cambia la fase y ejecuta `sem_post` $T - 1$ veces para despertar a los demás hilos.

Los hilos que esperaban consumen las señales con `sem_wait`. En caso de que una señal del sistema interrumpa `sem_wait`, una función envolvente reintenta la operación hasta completarla. Si la barrera tiene solo un participante ($T = 1$), no realiza esperas.

Parte del código de la barrera:

```
my_phase = b->phase;
b->waiting++;
if (b->waiting == b->total) {//si todos los hilos han llegado a la barrera, se desbloquean y continúan ejecutando
    b->waiting = 0;
    b->phase = 1 - b->phase;//cambio de fase para la siguiente barrera
    is_last = 1;
}
(void)pthread_mutex_unlock(&b->mutex);

if (is_last) {//si es el último hilo en llegar a la barrera, desbloquea a todos los demás hilos
    for (i = 0; i < b->total - 1; i++) {
        (void)sem_post(&b->turnstile[my_phase]);
    }
} else {
    (void)sem_wait_nointr(&b->turnstile[my_phase]);//si no es el último hilo,debe esperar a que se desbloquee
}
```

2.4. Instantáneas, progreso y ciclo de vida

Mientras el hilo 0 realiza la copia de E_z para las instantáneas, los demás hilos pueden comenzar a actualizar H_x y H_y, ya que ambas tareas leen E_z sin generar conflictos de escritura. Posteriormente, todos los hilos se sincronizan en la primera barrera del siguiente paso.

El *callback* recibe un puntero a E_z, el cual solo es válido durante su ejecución. Si un módulo externo requiere conservar la instantánea, debe realizar una copia profunda de los datos antes de que el *callback* retorne.

Las consultas `STATUS` y `PEEK` pueden reflejar diferencias de paso, dado que el progreso se actualiza en cada iteración, mientras que la instantánea se toma cada k pasos.

Antes de iniciar el cálculo, los hilos trabajadores esperan una condición de inicio. La función `fdtd_run` espera a todos los trabajadores con `pthread_join` antes de destruir los recursos. La liberación de memoria de la malla se efectúa con `fdtd_free`.

2.5. Variante sin barreras

La opción de compilación `-DSIN_BARRERA` deshabilita las esperas en cada paso. Esto permite que un hilo actualice E_z mientras otro lee dicho campo o procesa valores magnéticos obsoletos. La ausencia de barreras genera condiciones de carrera reales que invalidan los resultados. Con las barreras activas, el resultado numérico con $T = 1, 2, 4, 8$ debe ser idéntico bit a bit.

2.6. Tiempo y métricas

El ejecutable `fdtd_bench` mide intervalos con `clock_gettime(CLOCK_MONOTONIC, ...)` e imprime los resultados en formato CSV. Las métricas reportadas incluyen:

- `init_s`: tiempo consumido por `fdtd_init`.
- `compute_s`: promedio del tiempo empleado por hilo en `update_h` y `update_e`.
- `barrier_wait_s`: promedio del tiempo de espera en barreras por hilo.

- `io_s`: tiempo dedicado por el hilo 0 a la ejecución de *callbacks*.
- `total_s`: tiempo de pared (*wall-clock time*) desde `fdtd_init` hasta `fdtd_result`.

`total_s` representa la duración real de extremo a extremo, incluyendo inicialización, cómputo, esperas, *callbacks* y obtención de resultados.

2.7. Arquitectura del servidor de red y protocolo de comunicación

Para permitir el acceso remoto al motor de simulación, se implementó un servidor de red concurrente bajo el modelo *Thread-per-Client* en C17 (`-std=c17 -Wall -Wextra -pedantic -pthread`). El servidor escucha en un puerto TCP configurable (por defecto, 8080) y gestiona las conexiones entrantes mediante sockets.

1. **Control de E/S y señales:** el servidor utiliza `poll()` con un tiempo de espera de 300 ms en las lecturas de los sockets. Esto evita bloqueos indefinidos y permite monitorear continuamente la variable de control `running`, modificada de manera atómica ante una señal `SIGINT` (interrupción por teclado) para iniciar un apagado ordenado.
2. **Protocolo de texto formateado:** el módulo `protocolo.c` analiza las líneas entrantes mediante `ssize_t` y valida estrictamente los límites y la cantidad de argumentos. Los comandos soportados son `SUBMIT`, `STATUS`, `PEEK`, `RESULT` y `SHUTDOWN`. Las respuestas del servidor siguen un formato estricto terminado en salto de línea (`\n`): `OK <id>`, `BUSY`, `ERROR <motivo>`, `QUEUED <pos>`, `RUNNING <paso>/<total>`, `DONE <segs>`, `VALUE <paso><valor>`, `RESULT <suma><archivo>` y `BYE`.
3. **Transferencia completa (`send_all`):** para evitar escrituras parciales por fragmentación TCP, se implementó la función auxiliar `send_all`, que garantiza que todos los bytes de la respuesta se transmitan antes de continuar.
4. **Concurrencia por cliente:** cada cliente aceptado por `accept()` genera un hilo POSIX independiente en modo desacoplado (`pthread_detach`). Los contadores de conexiones aceptadas, conexiones atendidas e hilos activos se protegen mediante un `pthread_mutex_t` (`stats_mutex`).
5. **Cierre ordenado:** al recibir la orden `SHUTDOWN` o una señal `SIGINT`, el servidor deja de aceptar nuevas conexiones, cierra el socket de escucha y espera, mediante lecturas no bloqueantes con `nanosleep`, a que todos los hilos activos de clientes terminen antes de liberar los módulos del sistema y destruir los mutexes.

2.8. Gestión concurrente de los trabajos

Los trabajos recibidos por el servidor se administran con una cola, un gestor de estados y un despachador. Cada módulo cumple una función durante la ejecución de las simulaciones.

- **Cola de trabajos (`queue.c`):** utiliza una estructura circular FIFO con capacidad limitada. Un mutex protege sus operaciones y un semáforo señala la disponibilidad de trabajos. Si la cola está llena, el servidor responde `BUSY`.
- **Gestor de trabajos (`job_manager.c`):** asigna un identificador a cada solicitud y registra su estado (`QUEUED`, `RUNNING`, `DONE` o `ERROR`), progreso, tiempo y resultado. El acceso a estos datos se protege con un mutex.
- **Despachador (`dispatcher.c`):** toma los trabajos de la cola, ejecuta el motor FDTD y recibe el progreso mediante *callbacks*.
- **Estado compartido (`shared_state.c` y `rwlock.c`):** conserva la matriz E_z más reciente del trabajo actual. El candado permite varias lecturas simultáneas y acceso exclusivo para actualizar los datos.
- **Instantáneas (`snapshot.c`):** utiliza un búfer circular limitado y un hilo escritor que guarda los archivos binarios. Los semáforos coordinan los espacios y los datos disponibles. Antes de marcar un trabajo como `DONE`, se espera la escritura de la instantánea final.
- **Apagado (`system.c`):** cierra la cola, espera al despachador y al hilo escritor y, luego, libera los recursos.

3. Pruebas y evidencia local

3.1. Ambiente de pruebas

Las pruebas se realizaron en un procesador AMD Ryzen 3 3100 (4 núcleos físicos, 4 hilos visibles y 1 socket) sobre KVM, en un entorno virtualizado. La compilación se efectuó con `gcc -std=c17 -Wall -Wextra -pedantic -O2 -pthread`, sin generar advertencias ni errores.

Prueba	**Caso**	**Local**	**ECCI/KABRE**
Barrera	8 hilos, 1000 rondas	OK	Pendiente
Referencia	8×8, 5 pasos, 1/4/6 hilos	OK	Pendiente
Caso manual	1 paso; solo la fuente modifica E_z	OK	Pendiente
Reparto irregular	61/4, 61/7, 10/8, 13/11, 17/3	OK	Pendiente
Simetría	Ejes y diagonal, 1 y 4 hilos	OK	Pendiente
Bordes PEC	47×53 con 3 hilos, $E_z = 0$	OK	Pendiente
Determinismo	2/3/4/8 hilos idénticos bit a bit a 1 hilo	–	Pendiente
Sin barreras	20 corridas, 512×512, 300 pasos	20/20 fallan	Pendiente
Pruebas de protocolo	Análisis de comandos válidos, inválidos y vacíos	OK	Pendiente
Pruebas del servidor de red	Múltiples clientes concurrentes mediante `nc`/`fdtd_client`	OK	Pendiente
Apagado ordenado	Cierre con `SHUTDOWN` y `SIGINT` (Ctrl+C), sin fugas	OK	Pendiente
Cola FIFO	Orden, capacidad, posición y cierre	OK	Pendiente
Gestor de trabajos	Registro, estados, progreso y resultado	OK	Pendiente
Lectores-escritores	Lecturas simultáneas y escritura exclusiva	OK	Pendiente
Instantáneas	Archivo binario, encabezado y valores E_z	OK	Pendiente

Los registros completos están en `test_local.log` y `test_race_local.log`. La prueba de determinismo compara E_z, H_x y H_y con `memcmp`. Con `-DSIN_BARRERA`, las 20 ejecuciones detectaron diferencias, lo que evidencia la necesidad de las barreras. El resultado paralelo es igual al serial, salvo por diferencias mínimas de redondeo. La simetría se verifica con una malla cuadrada y una fuente centrada. Los bordes PEC se verifican comprobando que $E_z = 0$ en todo el contorno después de la simulación.

4. Análisis de resultados y rendimiento

4.1. Análisis del rendimiento paralelo

La serie principal de pruebas se ejecutó en `bench_E1_1000.csv`, con 5 repeticiones para $T = 1, 2, 4, 8$ hilos, sobre una malla de 2048×2048 celdas y 1000 pasos de tiempo, sin extracción de instantáneas.

T	Mediana (s)	Rango (s)	$S(T)$	$E(T)$	$e(T)$
1	21.5706	20.4140–24.3467	1.000	1.000	–
2	17.9586	16.8236–19.5935	1.201	0.601	0.666
4	17.5525	14.4874–19.4869	1.229	0.307	0.752
8	16.1455	14.5187–19.2256	1.336	0.167	0.712

La mediana se calcula ordenando las cinco observaciones y tomando la del centro. El rango es la diferencia entre el máximo y el mínimo, expresada aquí mediante sus extremos. La aceleración es $S(T) = T(1)/T(T)$, la eficiencia es $E(T) = S(T)/T$ y $e(T)$ es la fracción serial efectiva de Karp–Flatt:

$$e(T) = \frac{1/S(T) - 1/T}{1 - 1/T}.$$

Variabilidad: los rangos observados son amplios. Por ejemplo, con $T = 4$, el tiempo varía entre 14.48 s y 19.48 s. Esto puede deberse a que las pruebas se ejecutaron sobre una máquina virtual (KVM), donde los cuatro núcleos visibles no son necesariamente exclusivos y el hipervisor puede introducir ruido en las mediciones.

4.2. Estimación de la fracción paralelizable y ley de Amdahl

A partir de la ejecución con un hilo ($T = 1$), la mediana de `compute_s` fue 21.5634 s y la de `total_s` fue 21.5706 s. La fracción paralelizable estimada es:

$$p = \frac{21.5634}{21.5706} \approx 0.9997.$$

Aplicando la ley de Amdahl:

$$S_A(T) = \frac{1}{(1-p) + p/T}.$$

Para $T = 4$, Amdahl predice $S(4) \approx 3.99$, mientras que el valor medido fue de 1.229. Para $T = 8$, Amdahl predice 7.96 y se midió 1.336.

4.3. Discusión de resultados

El mejor tiempo mediano se obtuvo con $T = 8$ hilos (16.1455 s), pero la diferencia con $T = 4$ (17.5525 s) es pequeña y los rangos se solapan. En general, agregar hilos reduce el tiempo de cómputo, pero no de forma proporcional.

El análisis de las etapas ayuda a explicar este comportamiento. El tiempo de cómputo por hilo disminuyó de 21.563 s ($T = 1$) a 7.653 s ($T = 8$), lo cual confirma que la paralelización del cálculo funciona. Sin embargo, la espera en barreras se incrementó drásticamente, de 0.0015 s a 8.46 s. A partir de $T = 4$, las barreras representan una fracción considerable del tiempo total y absorben parte de los beneficios de la paralelización.

El aumento de la métrica de Karp–Flatt $e(T)$ de 0.666 ($T = 2$) a 0.752 ($T = 4$) indica que el cuello de botella está relacionado con los sobrecostos de sincronización, la saturación del ancho de banda de memoria y el efecto de sobresuscripción al ejecutar ocho hilos en un entorno de cuatro núcleos físicos.

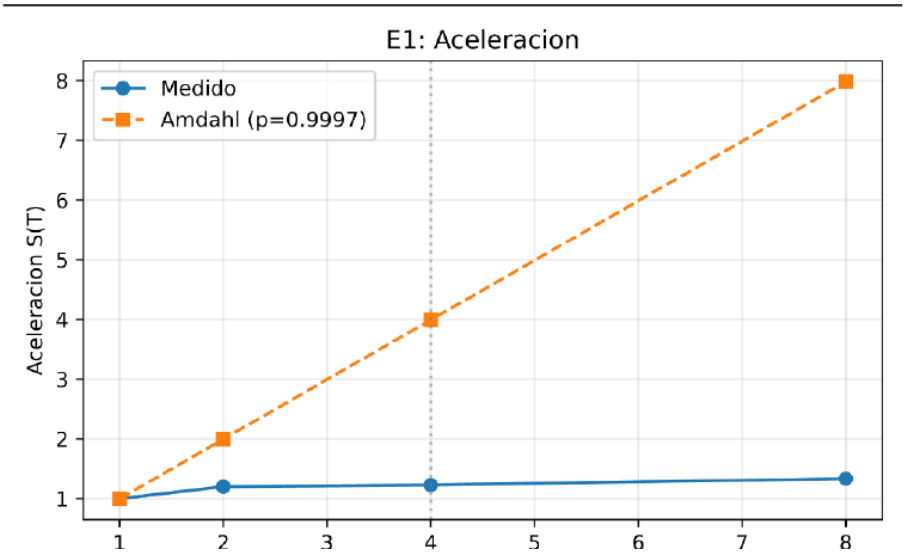


Figura 2: E1 (2048 × 2048, 1000 pasos): aceleración medida frente a la predicha por Amdahl. La línea vertical marca las cuatro CPU visibles.

La eficiencia cae de 1.00 con un hilo a 0.60 con dos, 0.31 con cuatro y 0.17 con ocho. Esto significa que cada hilo adicional aporta menos al rendimiento total, hasta el punto de que, con ocho hilos, cada uno aporta solo el 17 % de lo que aportaba el hilo único.

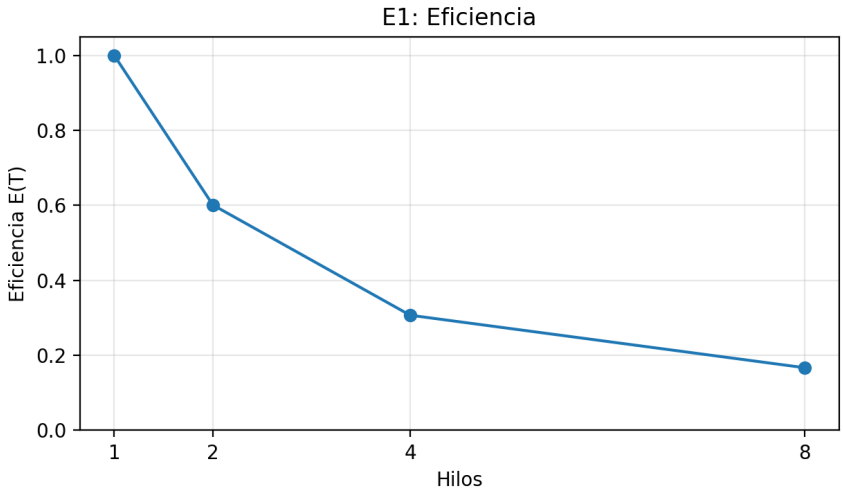


Figura 3: E1: eficiencia por hilo $E(T) = S(T)/T$.

La fracción serial efectiva de Karp–Flatt crece de 0.666 con dos hilos a 0.752 con cuatro hilos. Si solo hubiera una fracción serial pura, este valor sería constante. Como aumenta, indica que el deterioro no se debe únicamente a la parte serial del algoritmo, sino también a sobrecostos que crecen con el número de hilos: sincronización en barreras, saturación de memoria y sobresuscripción.

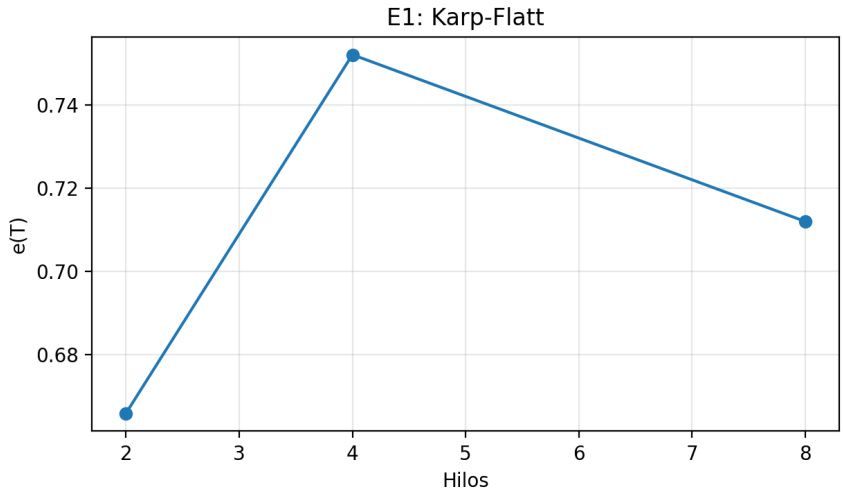


Figura 4: E1: fracción serial efectiva de Karp–Flatt.

4.4. Escenario E3: malla pequeña

Para contrastar el efecto del tamaño de la malla, se ejecutó el escenario E3 con una malla de 256×256 y 1000 pasos de tiempo. Los resultados se muestran a continuación:

T	Mediana (s)	Rango (s)	S
1	0.1519	0.1458–0.1620	1.000
2	0.1432	0.1229–0.2269	1.061
4	0.2136	0.2049–0.2674	0.711
8	0.2163	0.2124–0.3610	0.702

A diferencia de E1, en E3 la paralelización empeora el rendimiento. Con $T = 4$ y $T = 8$, el tiempo total es mayor que con un solo hilo (aceleración menor que 1). La razón es que, en una malla pequeña, el trabajo por hilo es tan corto que el costo de las dos barreras por paso supera la ganancia del cómputo paralelo. Esto confirma que, sin suficiente trabajo por hilo, las barreras dominan el tiempo de ejecución.

tiempo total es mayor que con 1 solo hilo (speedup menor que 1). La razón es simple: en una malla pequeña, el trabajo por hilo es tan corto que el costo de las dos barreras por paso supera la ganancia del cómputo paralelo. Esto confirma lo esperado en el enunciado: sin suficiente trabajo por hilo, las barreras dominan el tiempo de ejecución.

Figura 4. E3 (256×256, 1000 pasos): Con poco trabajo por hilo, el speedup medido es menor que 1 a partir de T=4.

5. Conclusiones

1. Las evaluaciones experimentales confirman el correcto funcionamiento del motor de cálculo FDTD 2D. Se comprobó la equivalencia numérica entre la solución paralela y la referencia serial, la simetría espacial, la preservación de los bordes PEC y el determinismo absoluto bit a bit en las ejecuciones multihilo.
2. Las pruebas con la opción -DSIN_BARRERA resultaron en un 100% de fallos numéricos (20 de 20 corridas), evidenciando que las barreras de sincronización por fase son estrictamente necesarias para prevenir condiciones de carrera durante la actualización de los campos magnéticos y eléctricos.
3. En el escenario E1 (malla grande), la mayor aceleración se alcanzó con T=8 hilos (S=1.336), aunque muy lejos de la predicción de la Ley de Amdahl (7.96). El cuello

Figura 5: E3 (256×256, 1000 pasos): con poco trabajo por hilo, la aceleración medida es menor que 1 a partir de $T = 4$.

5. Conclusiones

1. Las evaluaciones experimentales confirman el funcionamiento del motor de cálculo FDTD 2D. Se comprobó la equivalencia numérica entre la solución paralela y la referencia serial, la simetría espacial, la preservación de los bordes PEC y el determinismo bit a bit en las ejecuciones multihilo.

2. Las pruebas con la opción `-DSIN_BARRERA` resultaron en un 100 % de fallos numéricos (20 de 20 corridas), lo que evidencia que las barreras de sincronización por fase

son necesarias para prevenir condiciones de carrera durante la actualización de los campos magnéticos y eléctricos.

3. En el escenario E1 (malla grande), la mayor aceleración se alcanzó con $T = 8$ hilos ($S = 1.336$), aunque quedó muy lejos de la predicción de la ley de Amdahl (7.96). El cuello de botella no es la fracción serial, sino la latencia de sincronización en barreras y la contención en el acceso a memoria, como lo demuestra el comportamiento de la métrica de Karp–Flatt.

4. En el escenario E3 (malla pequeña), se comprobó que la paralelización puede ser contraproducente. Con $T = 4$ y $T = 8$, la aceleración fue menor que 1 (0.711 y 0.702). Esto evidencia que el beneficio de paralelizar depende de que exista suficiente trabajo por hilo para amortizar el costo de la sincronización.

5. El diseño y la implementación de la capa de red bajo el modelo *Thread-per-Client* y el protocolo basado en líneas de texto permiten separar la recepción de comandos de E/S y la ejecución matemática. La utilización de `poll()` con tiempo de espera, la transmisión completa de respuestas con `send_all` y la captura de señales `SIGINT` contribuyen a un servidor estable, tolerante a errores de sintaxis y capaz de realizar un apagado ordenado sin pérdida de recursos ni fugas de memoria.

6. La integración del gestor de trabajos, el despachador y el búfer de instantáneas permitió que el servidor organizara las solicitudes y ejecutara las simulaciones de forma ordenada. El uso de mutexes, semáforos y lectores-escritores ayudó a evitar conflictos al trabajar con datos compartidos. Además, las pruebas realizadas permitieron comprobar el funcionamiento de estos módulos en diferentes situaciones.

Enlace al repositorio: https://github.com/JavierChacon16/Proyecto01_server_CI0117